The Data Layer

Part 2 of 5: EF Core, the schema, the interceptors, and migrations

FILES: MODELS/ · DATA/APPDBCONTEXT.CS · DATA/*INTERCEPTOR.CS · REPOSITORIES/ · MIGRATIONS/

1. EF Core in five minutes

Entity Framework Core is an **ORM** — Object-Relational Mapper. It lets you write C# instead of SQL, and it writes the SQL for you.

DbContext and DbSet

`AppDbContext` is your session with the database. Each `DbSet<T>` on it is one table:

```
public DbSet<Student> Students => Set<Student>();
```

`_context.Students.Where(s => s.GraduationDate == null)` becomes `SELECT ... FROM "Students" WHERE "GraduationDate" IS NULL`. Crucially, that translation happens **lazily**. Until you call `ToListAsync()`, `FirstOrDefaultAsync()` or similar, you are holding an `IQueryable` — a description of a query, not results. This is why `Filter()` and `InRange()` helpers in the repositories can add `.Where(...)` clauses one at a time: nothing has run yet, so it all collapses into one SQL statement at the end.

The change tracker

This is the concept most worth understanding, because two bugs in this project came from it.

When EF loads an entity it normally keeps a reference to it and remembers its original values. This is **tracking**. Later, `SaveChangesAsync()` compares current values with the originals and issues `UPDATE` statements for whatever differs. You never write an `UPDATE`; you mutate the object and save.

```
// tracked -- EF is watching this object
var student = await students.GetByIdAsync(id, ct);
student.GraduationDate = dto.GraduationDate;
await students.SaveChangesAsync(ct);   // EF writes the UPDATE
```

`AsNoTracking()` switches this off. The objects come back detached, EF forgets them immediately, and it is meaningfully faster and lighter. The rule in this codebase:

> **Read-only query → `AsNoTracking()`. Query whose result you intend to modify → tracked.** Every projection query in the repositories uses `AsNoTracking`; `GetByIdAsync` does not, because callers edit what it returns.

Get this wrong in the dangerous direction and you get a silent bug: mutate a no-tracking entity, call `SaveChanges`, and *nothing happens*. No error, no update. That is precisely why `StudentService.UpdateAsync` loads the `User` through `userRepository.GetByIdAsync` (tracked) rather than through a no-tracking "student with user" query.

One DbContext per request

`AddDbContext` registers it as **scoped**, meaning one instance per HTTP request, shared by every service and repository in that request. Two consequences that look surprising until you know this:

- Calling `SaveChangesAsync` on *any* repository saves *all* pending changes, including ones made through a different repository. They share the context.
- Multi-step writes are naturally atomic. `StudentService.CreateAsync` adds a `User` and a `Student`, then saves once — EF wraps that in a single transaction, so you can never end up with a login and no student.

2. Entities and the shape of the schema

Entities are plain classes in `Models/`. No logic, no attributes — all configuration lives in `AppDbContext.OnModelCreating`, which keeps the models readable.

Navigation properties, and their deliberate absence

A navigation property lets you walk a relationship in C#: `student.User.FirstName`. `Student` has them; `Class`, `Mark`, `Attendance`, `DiaryEntry` and `Enrollment` do not — they only carry raw `Guid` foreign keys.

That asymmetry explains a real inconsistency in the repositories, and you should be ready to defend it:

ENTITY HAS NAVIGATIONS?	HOW THE REPOSITORY READS IT	EXAMPLE
Yes	`.Include(...)` then AutoMapper	`StudentRepository`
No	Manual `join`, project straight into the DTO	`MarkRepository`

So `MarkRepository.Project` joins eight tables by hand to produce a `MarkListDto` containing the student's name, the class name, the subject name and the teacher's name. It is verbose, but it is one SQL query and it fetches exactly the columns the DTO needs.

3. The composite-key technique

This is the cleverest part of the schema and the most likely thing to be asked about. It is worth being able to explain slowly.

The problem

Suppose `Enrollment` has two ordinary foreign keys: `StudentId → Students.Id` and `ClassId → Classes.Id`. Each is individually valid. But nothing stops a row pairing a student from School A with a class from School B. The database would happily accept it. You would be relying on application code to never make that mistake.

The solution

Give `Student` and `Class` an **alternate key** — a second uniqueness guarantee besides the primary key — on `(Id, SchoolId)`, then make `Enrollment`'s foreign key point at the *pair*:

```
modelBuilder.Entity<Student>().HasAlternateKey(s => new { s.Id, s.SchoolId });
modelBuilder.Entity<Class>().HasAlternateKey(c => new { c.Id, c.SchoolId });

modelBuilder.Entity<Enrollment>()
    .HasOne<Student>().WithMany(s => s.Enrollments)
    .HasForeignKey(e => new { e.StudentId, e.SchoolId })
    .HasPrincipalKey(s => new { s.Id, s.SchoolId })
    .OnDelete(DeleteBehavior.Restrict);
```

Now `Enrollment` carries its own `SchoolId`, and *the same value* must match the student's school and the class's school. A cross-school enrollment is no longer a bug to be avoided — it is a row PostgreSQL will refuse to insert.

> **The principle:** push invariants down into the database wherever the database can express them. Application code can be bypassed, forgotten, or written wrong in one of six services. A foreign key cannot.

The same trick, used harder

`Attendance` stores a `ClassId` that is, strictly speaking, redundant — you could reach the class through either the enrollment or the diary entry. It exists for exactly one reason:

```
.HasForeignKey(a => new { a.EnrollmentId, a.ClassId })   // → Enrollment(Id, ClassId)
.HasForeignKey(a => new { a.DiaryEntryId, a.ClassId })   // → DiaryEntry(Id, ClassId)
```

Both keys include `a.ClassId`, the same column. So the enrollment's class and the lesson's class are forced to be identical. That is how “you cannot mark attendance for a student who is not in that lesson's class” becomes a database guarantee. Two independent foreign keys cannot express it; this can.

`Mark` pushes it furthest. Its foreign key into `ClassSubjectTeacher` spans five columns, forcing the mark's class, subject, teacher and school to match the teaching assignment exactly — so only the officially assigned teacher can grade that class-subject pairing. And two separate keys into `Class`, one on `(Id, SchoolId)` and one on `(Id, AcademicYearId)`, force the grading period into the same academic year as the class. A 2025/2026 class cannot take a mark from a 2024/2025 term.

4. Deletes: why almost nothing is deletable

Every relationship uses `DeleteBehavior.Restrict`. The database refuses to delete a row that anything else references.

This is a deliberate stance, not caution for its own sake. A school record is a legal and historical document. If deleting a class silently deleted its lessons, its attendance and its marks, a single mistaken click would erase a year of a child's record. So instead of deleting, the model *ends* things:

INSTEAD OF DELETING…	…YOU DO THIS	WHY IT WORKS
a user	set `User.DeactivatedAt`	Login stops; their marks still name a real person.
an enrollment	set `Enrollment.LeftAt`	The student's marks for that class survive the transfer.
a teaching assignment	set `ClassSubjectTeacher.EndedOn`	Past marks still point at whoever really taught then.
a student	set `Student.GraduationDate`	They leave the active roll, keep their history.

Filtered unique indexes

"Ending" creates a puzzle: how do you allow only *one active* assignment per class-subject, while still permitting old ended ones to sit alongside it? A plain unique index on `(ClassId, SubjectId)` would block the replacement.

```
modelBuilder.Entity<ClassSubjectTeacher>()
    .HasIndex(cst => new { cst.ClassId, cst.SubjectId })
    .IsUnique()
    .HasFilter("\"EndedOn\" IS NULL");
```

The index only covers rows where `EndedOn IS NULL`. Ended rows are invisible to it. So one active assignment is enforced, and history accumulates freely. `Enrollment` uses the identical trick with `LeftAt`.

5. Multi-tenancy, part one: global query filters

Every school-scoped entity implements a one-property marker interface:

```
public interface ISchoolScoped { Guid SchoolId { get; } }
```

And each gets the same filter, applied through one helper in `AppDbContext`:

```
private void ScopeToSchool<T>(ModelBuilder modelBuilder) where T : class, ISchoolScoped =>
    modelBuilder.Entity<T>().HasQueryFilter(e => e.SchoolId == currentUser.SchoolId);
```

From then on, **every query EF generates for that entity silently gains** `AND "SchoolId" = @currentSchool`. Not most queries — every one, including nested ones inside joins. You cannot forget it, because you never write it.

Two details that matter:

- `currentUser.SchoolId` is `Guid?`. If there is no school claim — an anonymous request, or a SuperAdmin — the comparison is against `NULL` and matches nothing. **The failure mode is "see nothing", never "see everything".** That is the correct direction for a security default to fail in.
- The filter is escapable with `.IgnoreQueryFilters()`. That is intentional — some operations genuinely must cross schools — but every such call is a deliberate, greppable decision. There are nine in the

codebase and Part 3 lists them all.

6. Multi-tenancy, part two: the write-side interceptor

Query filters only protect *reads*. Nothing in them stops you constructing a `Subject` with someone else's `SchoolId` and saving it. That gap is closed by an interceptor.

An **interceptor** is a hook EF Core calls at defined moments in its own pipeline. `SchoolScopeInterceptor` hooks "just before saving":

```
private void Validate(DbContext? context)
{
    if (context is null || currentUser.IsSuperAdmin)
        return;

    var schoolId = currentUser.SchoolId;

    foreach (var entry in context.ChangeTracker.Entries<ISchoolScoped>())
    {
        if (entry.State is not (EntityState.Added or EntityState.Modified or EntityState.Deleted))
            continue;

        if (entry.Entity.SchoolId != schoolId)
            throw new InvalidOperationException(...);
    }
}
```

It walks everything about to be written, and refuses anything not in your own school. Note the SuperAdmin exemption on line 2: a SuperAdmin has no school, so without it they could never create an `Admin` (which *is* school-scoped). One `if` replaced what used to be a much more elaborate suppression mechanism.

> **Read this pairing as one idea.** Query filter = you cannot *see* another school. Interceptor = you cannot *write* into another school. Neither alone is sufficient; together they are the tenancy boundary.

7. The audit interceptor

The second interceptor writes the audit trail. It hooks the same moment and adds one `AuditLog` row per changed entity *to the same save*:

```
var changes = context.ChangeTracker.Entries()
    .Where(e => e.Entity is not AuditLog
                && e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)
    .Select(e => $"{e.State} {e.Entity.GetType().Name} ({Key(e)})")
    .ToList();

if (changes.Count == 0) return;

context.AddRange(changes.Select(action => new AuditLog { ... }));
```

Three things to notice:

- `e.Entity is not AuditLog` stops it auditing itself, which would recurse forever.
- The log rows join the **same transaction** as the change they describe. Either both commit or neither does — an audit trail that can silently go missing is worse than none.
- It only works because ids exist before saving. See the next section.

8. How ids are generated

Every primary key is a `Guid`, generated **by EF in C#, at the moment you call `AddAsync`** — not by the database.

```
b.Property<Guid>("Id")
    .ValueGeneratedOnAdd()
    .HasColumnType("uuid");
```

The timing is the whole point:

```
new DiaryEntry() → Id is empty AddAsync(entry) → Id is set, right here SaveChangesAsync() → the id you already have is written
```

This matters because `DiaryEntryService.CreateAsync` creates a lesson and, in the same breath, one `Attendance` row per enrolled student — each needing `DiaryEntryId`. Because `Attendance` has no navigation property back to `DiaryEntry`, EF cannot wire the id up automatically. With client-generated ids the id is simply already there, and the whole operation is one atomic save.

An earlier version had PostgreSQL generate ids via `gen_random_uuid()`. Ids then did not exist until after `SaveChanges`, which forced a manual `Guid.NewGuid()` in the service and a two-phase audit interceptor that wrote its rows in a *second* transaction. Moving generation to the client removed both problems.

EF's generator produces increasing rather than random Guids, which keeps B-tree index inserts appending at the end instead of scattering across pages. If you assign an `Id` yourself, EF uses yours — it only generates when the property is still `Guid.Empty`.

9. Optimistic concurrency

```
modelBuilder.Entity<Student>().Property<uint>("Version").IsRowVersion();
```

This is a **shadow property** — it exists in the model and the database but not on the C# class. Npgsql maps it to PostgreSQL's system column `xmin`, which changes on every row update.

It protects against lost updates. Two admins open the same student; both save. Without this, the second silently overwrites the first. With it, EF includes the original version in the `WHERE` clause, the second `UPDATE` matches zero rows, and EF throws `DbUpdateConcurrencyException`.

Honest caveat you should state if asked: that exception is not currently caught, so it would surface as a 500. Mapping it to a 409 Conflict is a genuine and easy improvement.

10. Migrations

A migration is a generated C# file describing a schema change, so the database can be rebuilt reproducibly from source control.

```
dotnet ef migrations add SomeDescriptiveName   # generate from model changes
dotnet ef database update                      # apply pending migrations
dotnet ef migrations remove                    # undo the last, if unapplied
```

`Migrations/AppDbContextModelSnapshot.cs` records what EF believes the schema currently is; the next `add` diffs your model against it. Never hand-edit it.

> **The rule:** a migration that has been applied to a shared database is history — add a new one to change things. A migration that exists only on your machine can be deleted and regenerated. This project's `InitialCreate` was regenerated after the id-generation change, which was safe precisely because it had never been applied anywhere.

11. The repository layer

`Repository<T>` is a small generic base with five members. Two are worth explaining:

```
public async Task<T?> GetByIdAsync(Guid id, CancellationToken ct) =>
    await _context.Set<T>().FirstOrDefaultAsync(e => EF.Property<Guid>(e, "Id") == id, ct);
```

`EF.Property<Guid>(e, "Id")` reaches a property by name, because `T` is generic and has no compile-time `Id`. It is written as a query rather than the more obvious `DbSet.Find()` deliberately: `Find` can return an entity straight from the change tracker without ever consulting the school query filter. For a multi-tenant app that is exactly the wrong shortcut.

```
public async Task<bool> SaveChangesAsync(CancellationToken ct) =>
    await _context.SaveChangesAsync(ct) > 0;
```

`SaveChangesAsync` returns the number of rows affected; this reports whether anything happened.

Each entity then gets its own repository for queries the generic base cannot express. Be ready for the obvious challenge — *"EF Core is already a repository and unit of work, so why wrap it?"* — and Part 5 gives you an honest answer.